



I want you to act as a teacher with good knowledge of AI/ML and explains complex topics easily, but must include the necessary terminologies. Now I want you to create Notes on these topics, I want you to include everything related to which is necessary with the interview perspective. Also include a few practice questions mainly coding questions and projects related to that particular topics.

Raw text:

Learn yourself stuff:

1. anova framework: analysis of variance. SSR=sum of squared regressor, SSE=sum of square error, SST=sum of square total.

- SSE: sum of errors
- SSR: From the predicted point to average
- SST = SSE + SSR
- Why we need anova?
 - = To find regression model's accuracy.
- $R^2 = 1 - SSR/SST$
- Adjusted R^2
- Both the of their range is always between 0 to 1.
- R squared and adjusted r squared is a performance measure of regression model.
- * Type 1 type 2 error comes in classification models
- * R squared and Adjusted R squared comes under regression models.
- =Machine Learning===
- Training phase, testing phase, validation phase

- Model?
- Traditional model: input + input == output
- ML model: Input, output (inputs only) == logic that can generate output to different input.
- Supervised data:
 - regression: Dependent variables are continuous
 - classification: Dependent variables are binary.
- Unsupervised data:
 - Clusters: no dependent variable here, just groups.

(historical: Seen data, future data: unseen data)

- Reinforcement learning:

- Pipeline for ML model:

1. Gathering data
2. Preparing that data
3. choosing a model
4. Training
5. Evaluation
6. Hyperparameter Tuning
7. Prediction
8. ML ops
9. CI/CD Pipeline

=Data preprocessing pipeline=

1. Sql query to get the raw data or client shared the data or other team shared the data.
2. We divide the data into X & y where x is independent var and y is dependent var.
 - x is divided into x train and x test
 - y is divided into y train and y test
3. then we train the data using X_train + Y_train, then the model will get generated.
4. How to divide the data?
 - 70-30, 80-20 or 75-25%.

Data analyst: work on structured data

Data engineer: work on unstructured data.

=Spyder App=

- Used for training ML models.
- Scikit learn-
- It is built on top of Data structures required for machine learning. It also include Math, linear algebra and stats.
- How to import: from sklearn.
- Impute, is a transformer
- fit and transform
 - Parameter tuning & Hyperparameter

14-08-22025

Regression →

- Simple linear regression: $y=mx+c$
-

Classification →

Clustering →

=18-8-2025=

****Bias & Variace**

-trained the data == Bias

-Test the data == Variance

Bias = 94, variance - 34 (High bias and low variance) == Underfitting

Bias = 40, variance = 89 (Low bias and high variance) == Overfitting

Bias = 94 & Variance = 82 (Low Bias & Low variance) == Best fit model

This is called the Bias-Variance tradeoff.

****Overfitting and Underfitting****

Overfitting: Training the model with a lot of attributes is called overfitting. Overfit model = Misclassification

- Technique to reduce overfitting

1. PCA (Principle Component analysis)
2. Regularization technique (lasso & ridge)
3. ensemble learning
4. Cross Validation
5. Drop out the neurons
6. Business Knowledge

****Underfitting****

- Training the model with a very few variables/attributes is called underfitting. This also leads to Misclassification.

Techniques to reduce underfitting

- By adding more relevant attributes while training the data.

****Stats for ML**** in 02_linear_regression.py file in August>Practical

---19-8-2025---

Simple linear model/regression COMPLETED

****Multiple linear regression Model 20-08-2025****

Formula: **$y = m_1 \times x_1 + m_2 \times x_2 + m_3 \times x_3 + c$**

- Backward Elimination
- Recursive Feature Elimination
- P-value
- statsmodels

Linear REgression

- SLR
- MLR
- Regularization
- Gradient Descent, SG, BGD
- Time Series

Non Linear Regression

- Polynomial
- Support Vector machine
- Decision Tree

- Random Forest
- xgboost regression
- lgb light gradient boost regression
- ann artificial neural network regression

=Overfitting=

- Low bias high variance
- If you build the model with 100 attributes(features) the model will Overfit, so take only relevant features.
- If coefficient of independent variable is very high, also leads to overfitting (Regularization)

Underfitting=

- high bias low variance

Low bias, Low variance

- Best fit model.

=Regularization=

- To reduce high coef to low coef & scale down overfitting regularization is used
- Feature selection Techniques:

1. lasso regression(l1)(powerful, why?)(feature elimination)

$$\text{Lasso} = \text{Loss} + \alpha ||w||$$

(Penalty)

2. ridge regression(l2):

$$\text{Ridge R} = \text{Loss} + \alpha ||w||^2$$

(Penalty)

3. elastic net regression(l1+l2)

- Inherits few concepts from l1 & l2

=Gradient Decent=

(read the jupyter notebook: 21_22082025_Gradient_decent)

===== DONE TILL NOW =====

Feature selection technique

1. Business understanding
2. p-value, rfe, backward elimination
3. lasso regularization
4. decision tree
5. pca

Feature engineering

1. eda technique
2. variable identification
3. missing variable treatment
4. imputation or transformer
5. outlier, univariate, bivariate, multivariate

Feature Scaling

1. Standardization
2. Normalization

ML concepts:

Linear: Simple linear, Multiple linear, gradient descent, batch gradient descent, l1, l2

Non Linear: polynomial reg, support vector reg, knn, decision tree, random forest, etc. (these are remaining).

=26-06-2025=

Support Vector Regressor (used for regressor) (SVM is used for Classification)

- Best Fit line = Hyperplane
- $y=wx+b$
- One side of the plane (line) is positive and another is negative, and it is called distance between 2 support line also called marginal distance.
- Decision boundry == svr line
- Final equation: $-a < y-mx + B < + a$
- Difference between the margins should be Maximum, therefore it's called maximum margin difference.

=KNN=

Distance Matrix

- Euclidian distance (we can apply this on most of the algorithms)
= Shortest distance
- Manhattan distance
= Farthest distance
- Cosine Distance
= distance amongst the words, used in NLP, vector, llm

=28th august=

prev: Poly: Increased the degrees of independent variable

SVR: max marginal distance

KNN: distance matrix (euclidean distance & manhattan distance)

today:

=Decision tree & Random Forest=

- Gini index

- entropy
- max depth
- pruning
- leaf node
- root node

- How to decide root node, leaf node, purity, etc.

Random Forest:

Group of decision trees is called random forest

Ensemble learning

- Bagging
 - Random Forest
- Boosting

=2-9-2025=

Classification Regression

A) Performance measure of regression: R-squared, Mean absolute error, Mean squared error, RMSE, confusion matrix.

Algorithms:

1. Logistic Reg
2. SVM
3. DT
4. KNN
5. Random Forest
6. xgboost
7. lgbm
8. naive bayes
9. ANN classifiers

=Confusion matrix=

Formula =

Accuracy = $(TP + TN) / \text{Total}$

Error = $(FP + FN) / \text{Total}$ Or $1 - \text{accuracy}$

Precision

Recall

$f1 \text{ score} = 2 * (\text{precision} * \text{recall}) / (\text{precision} + \text{recall})$

- Why to use all these matrix? how does it affect the model?

=Logistic Reg=

- Logistic reg is both classification and regression model

- Sigmoid function is applied to deal with the outlier.
- Logistic regression is also called as maxent classifier

=PCA= 4-9-2025=====

PCA(Principal component analysis)

- used for clustering
- minimizes the larger coefficient
- WE can create multiple PCs from PCA

How to find out best PC based on highest eigen vector

Logistic Regression in PCA.

- Explained Variance Ratio

KNN Regression- take the mean of the nearest point based on k

=KNN Classification=

- Here, we take based on maximum number of points near k (something like that)
- Distance Matrix
 - Euclidian distance
 - Manhattan distance
 -

What's balanced data? what's imbalanced data?

How to convert imbalanced to balanced data?

- Oversampling- Increasing the minority class (1) to (30)
- Under sampling- Decreasing the majority class (99) to 70
- You can play with the values (30, 70), (20,80), etc.
- This technique is Synthetic Minority Oversampling Technique (SMOTE) also called as Resampling.

=====

OUTLIERS DO NOT IMPACT LOGISTIC REGRESSION because of Sigmoid curve.

OUTLIERS IMPACT KNN MODEL

OOPS

=====

- class
- methods
- object

- encapsulation
- polymorphism
- abstraction
- inheritance

=6-9-25=

- Linear SVM
- Non linear SVM
- support vector
- hyperplanes
- marginal dist
- linear seperable
- non linear separable
- Kernal: sigmoid, poly, precomputer, linear, rbf

=Naive Bayes algo=probability

Concepts to understand:

1. Conditional Probability:
2. Bayes or Bayesian Theorem
3. Naive bayes algorithm
4. Realtime implementation of naive bayes
5. types of naive bayes
6. practical

What is naive?

- 2 independent events $p(a)$ & $p(b)$ is called naive in naive bayes.

$$P(A|B) = P(A \cap B) / P(B)$$

===Baysian Theorem

- Posterior prob
- Likelihood
- Marginal prob
- Prior prob

====SVM=

Linear SVM (SVR (maybe)

- Here we can easily distribute the devide the classes with single line (Hyperplane) and increase the margincal distance, it'll work.

Non linear SVM

- It's circular, cos we cannot divide the data with single like(hyperplane).
- So here we convert this 2d data to 3d data using kernel functions. (sigmoid, poly, precomputer, linear, rbf, etc.)
- they convert non linear data to linear
- Here we will convert Low dimension(non linear) to high dimension (Linear)

Learn:

- Conditional Probability
8-9-2025=
SQL integration with Python
9-9-2025===
- Logistic and Naive bayes are probability algorithm
- Concepts in Naive bayes

1. Conditional Probability
2. Bayes theorem
3. naive bayes algorithm
4. realtime naive bayes (implementation)
5. Type of naive bayes
6. Practical

=====

3 types of Naive Bayes:

1. BernoulliNB Classifier (works on Binary)
 2. Gaussian Naive Bayes (works on Binary)
 3. Multinomial Naive bayes (Do not apply on Binary dependent variable, also do not apply when negative values (recheck))
- Check the accuracy of every model
 - once with standard scalar
 - once with Normalizer
 - once without scaling
Then check the accuracy.

(Gaussian NB, tree algorithm does not require feature scaling)

10-9-2025=

TREE ALGORITHM

1. Decision Tree regression
2. Decision tree classification
 - How to select a root node?
3. Steps to build decision tree

- Understand the dataset
- chose Dependent Variable and independent variable
- Gain of independent variable
Gain formula = Information Gain - Entropy
 - Gain (Age) = $ig - E(\text{Age})$
 - Gain (Competition) = $IG - E(\text{Competition})$
 - Gain (

Topics to learn

- Entropy, gain, information gain, purity split, leaf node, root node
- Root node never dependant var, always independent variable
- if one side is 0, it's called purity split.
- Maximum gain will decide the root node
- Gini > Entropy
- Pruning = removing the irrelevant attributes
 - It helps to avoid overfitting

Crypto price prediction.

- yfinance library

==11-9-2025=

- bagging
- Xgboost (Churn Modelling dataset)

===12-9-2025

LightGBM (check doc on Kaggle)

-

"We care completed all ML classification algorithms"

- Techniques to reduce the overfitting
1. PCA
 2. ensemble
 3. regularization
 4. cross validation
 5. drop out the neuron (ANN, CNN, RNN, GNN)

=====

Cross Validation:

1. K fold cv:
 - here we split the data in equal number then take average
2. stratified cv
 - Here we split randomly and take average

Model Tuning:

1. Grid search cv
2. random search cv

---random---

bias vs hallucination

ROC - AUC Curve (graph of confusion matrix)

- x axis = FPR
- y axis = TPR

in auc roc curve we will get only classification not regression

AoC = Sensitivity

RoC = 1 - Specificity (What's the difference)

(roc = receiver operating characteristic

auc = area under curve)

=====

Machine Learning - Regression & Classification is completed

Clustering algorithms

1. PCA
2. K-means
3. Hierarchical clusters
4. dbscan cluster

Kimaya gambhe

Lumel technologies

Bharat software solutions

Amit kumar

=CLUSTER ALGORITHMS=15-9-2025=====

K-Means Clustering:

- Centroid = center point of the group
- Select any random point as centroid
- for K1 and K2 select the centroids
- Euclidean distance is used to calculate with datapoint is near to which cluster.
- Then calculate the new centroid (checkout the formula)
- How to calculate the number of K? use elbow method
 - WCSS = y axis, Number of clusters= x axis

Empirical probability distribution

Greedy k-means

=16-9-2025=

Hierarchical Clustering

- Dendrogram= connection between 2 points
 - Bottom to top approach
- Types:
1. Agglomerative cluster: using dendrogram (bottom to top)
 2. divisive cluster: divide the cluster (top to bottom approach)

Dbscan clustering:

- Density Based Spatial Clustering of Applications with Noise (full form for DBSCAN)
- Noise point = outlier
-

=ML is Completed=

=NLP=20-9-2025====

Checkout the Spyder files for code

- Bag of Words algorithm
- What is dummy variable and what is embedding?
 - What's the difference?
- Tfidf- formula, what it mean, why used, etc.
- word2vector:
 - Semantic Similarity Search
 - Library used: Gensim
 - Two parts: CBOW and SKIPGRAM

22-9-2025====

Word2Vec

- Used to understand the similarity between the words
 - e.g. king, male, kingdom, etc.

CBOW: Continuous Bag of Words

- It's an architecture used to predict the target word based on the context/surrounding words.

Skipgram

- It predicts the surrounding words based on the target words.

while coding, the hyperparameter is named as 'sg'

sg=0 = CBoW model

sg=1 = Skipgram model

These are not used much nowadays

--Word embedding algo we are completed--

Chunking: A paragraph split into section, token, small sentences is called as chunking. So simply splitting of the text data is called chunking.

Types of chunking:

1. Noun Phrase
2. Verb Phrase
3. Prepositional Phrase

2-9-2025=

SPACY = Advanced NLP. It's a library

=15-9-2025

Tweepy: Free api to extract the tweets

=Deep Learning=

frameworks:

1. TensorFlow
2. Pytorch (torch)
3. Keras,
4. yolo (lib name: ultralytics)
5. computer vision
6. istrm
7. gru
8. Mediapipe (mediapipe)
9. Langchain
10. Langgraph

DL is collection of neural networks

ANN

CNN

RNN

=26-9-2025=ML Ops===

Using mlflow library we can do ML Ops

- ML Ops is nothing but

Mostly use VS Code.

pip install keras torch opencv-python

=29-9-2025CI/CD==

- ML Model with continuous integration & continuous deployment.

Formula for new weight:

new weight = old weight - learning rate * derivative for loss / derivative old weight

weights are adjusted while you do the back propagation

Activation Function (in Deep Learning):

Activation function means...

Sigmoid:

We learnt this in logistic regression it was adjusting the outliers to reduce error

Sigmoid func always implement in output layer

Range = 0 to 1

It's 'S' curve

Won't work with negative value, it'll cap all negative to 0, that is not right

Calculates probability distribution

TanH:

Range is -1 to 1

Implemented in output layer only

Relu:

Rectified Linear Unit

range = 0 to inf

formula = $\max(x, 0)$ where x = input, so negative will be considered 0

Applied only to Hidden layer

Leaky Relu:

Range = -1 to inf

Softmax

Data = image, video, stream

Steps in CNN:

Convolution

Max pooling

Flattening

Full connection

Keras will convert the image to Arrays

Convolution

Convolution is a mathematical operation that allows merging of two sets of information.

In CNN, Convolution is applied to the input data to filter the information and produce a feature map

Striding Concept: Stride denotes how many steps we are moving in each steps in convolution.

By default it is one, We can observe that the size of output is smaller than input, means we lose the clarity.

e.g. input matrix = 6×6 , but the output will be 4×4

Therefore We use padding:

Stride denotes how many steps we're moving in each steps in convolution & by default it's one.

Padding creates extra space within an element. In contrast, margin creates extra space around

an element.

26th Sep 2025: MLOps

ML Ops stands for Machine Learning Operations

ML Ops support CI/CD

Collaborations b/w Data science and operations teams

Machine

Learning

DevOps

Data

Engineering

ML Ops

Key components

Version Control

Continuous integration

Model deployment

Model Monitoring: Performance checking, retraining if necessary

ML Ops workflow

Step 1: Data Collections - Gathering Data for training the model

Step 2: Model training -

Step 3: Model deployment

Step 4: Model monitoring

Step 5: Model Update: If performance degraded, retrain

ML Ops tools:

Mlflow

docker

tfx: Tensor flow extended

cloud platforms

Databricks

Kubeflow

6th October: Deep Learning & ANN

Forward + Backward propogation = 1 Epoch

Forward propogation = weight will generate

backward propogation = Weights are adjusted to match the y actual

activation function = applied on hidden layer to optimize the weights and accuracy

7th October: Vanishing Gradient Descent

When the forward prop weights gets equal to backward prop weights,

the epoch stops, cos old weights = new weights, that problem is VGD.

And the neural network model gets overfit. So to overcome this problem,

We kill the neuron means dropout a few neurons. Or

Regularization, high coef to low coef

Chain Rule:

To find the neuron causing vanishing gradient descent

8th October: CNN (Convolutional Neural network)

MAX POOLING:

This works by selecting the maximum value from every pool.

Max Pooling retains the most prominent features of the feature map and the returned image is sharper than the original image

Max pooling is a pooling operation that calculates the maximum value for patches of a feature map, and uses it to create a down sampled (pooled) feature map

It is usually used after a convolutional layer.

FLATTENING LAYER:

Flattening is used to convert all the resultant 2d arrays from pooled feature into a single long continuous linear vector

The flattened matrix is fed as input to the fully connected layer to classify the image

Flatten layer is used to make multi dimensional input 1d, commonly used in the transition from the convolution layer to the full connected layer.

FULLY CONNECTED LAYER:

A fully connected layer refers to a neural network in which each neuron applies a linear transformation to the input vector through a weights matrix. as a result, all possible connections layer to layer are present, meaning every input of the input vector influences every output of the output vector.

Fully connected layer are an essential component...

Convolution

Max pooling

Flatten layer

Fully connected layer

Now I'll create a comprehensive set of notes covering all the topics from the user's raw text with practice questions and coding problems.

Comprehensive Machine Learning & Deep Learning Notes

Prepared for Interview Excellence

This comprehensive guide covers fundamental to advanced concepts in Machine Learning and Deep Learning, with a focus on both theoretical understanding and practical implementation. These notes are designed to prepare you for technical interviews while providing deep insights into each algorithm and technique.

1. ANOVA Framework and Regression Metrics

1.1 Understanding ANOVA (Analysis of Variance)

ANOVA is a statistical framework used to evaluate regression model performance by partitioning the total variance in the data into explained and unexplained components. ^{[1] [2] [3]}

Key Components:

- **SST (Sum of Squares Total):** Measures total variability in the dependent variable
$$SST = \sum_{i=1}^n (y_i - \bar{y})^2$$
where y_i is the observed value and $\bar{y}$ is the mean. ^{[2] [4]}
- **SSR (Sum of Squares Regression):** Measures variability explained by the model
$$SSR = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$
where $\hat{y}_i$ is the predicted value. ^{[4] [2]}
- **SSE (Sum of Squares Error):** Measures unexplained variability (residual error)
$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$
 ^{[2] [4]}

Fundamental Relationship: $SST = SSR + SSE$ ^{[1] [4]}

This decomposition allows us to understand how much variance our model explains versus random error.

Performance Metrics:

R-squared (Coefficient of Determination):

$$R^2 = 1 - \frac{SSE}{SST} = \frac{SSR}{SST}$$

Range: , where 1 indicates perfect fit. R-squared represents the proportion of variance in the dependent variable that is predictable from the independent variables. ^{[5] [1] [2]}

Adjusted R-squared:

$$R^2_{adj} = 1 - \frac{(1 - R^2)(n - 1)}{n - k - 1}$$

where n is sample size and k is number of predictors. Adjusted R-squared penalizes the addition of irrelevant features, providing a more reliable measure for model comparison. ^[1]

Why use ANOVA? ANOVA helps determine whether a regression model provides a statistically significant fit to the data compared to a model with no predictors. The F-statistic from ANOVA tests the null hypothesis that all regression coefficients are zero. ^{[3] [6]}

1.2 Bias-Variance Tradeoff

The **bias-variance tradeoff** is fundamental to understanding model performance and generalization. ^{[7] [8] [9] [10]}

Bias: Error from incorrect assumptions in the learning algorithm. High bias leads to **underfitting**. ^{[8] [7]}

- Example: Using linear regression for non-linear data
- High training error and high test error

- Model is too simple to capture underlying patterns

Variance: Error from sensitivity to small fluctuations in training data. High variance leads to **overfitting**.^{[10] [7] [8]}

- Example: Training decision tree with unlimited depth
- Low training error but high test error
- Model memorizes noise instead of learning patterns

Total Error Formula:

$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$ ^[10]

Model States:

1. **Underfitting** (High Bias, Low Variance): Training accuracy = 94%, Test accuracy = 34%
[raw text]. Model is too simple.
2. **Overfitting** (Low Bias, High Variance): Training accuracy = 40%, Test accuracy = 89%
[raw text]. Model is too complex.
3. **Best Fit** (Low Bias, Low Variance): Training accuracy = 94%, Test accuracy = 82%
[raw text]. Optimal balance.

The goal is finding the sweet spot where both bias and variance are minimized, leading to good generalization on unseen data.^{[9] [8] [10]}

1.3 Techniques to Reduce Overfitting

1. PCA (Principal Component Analysis): Reduces dimensionality by projecting data onto principal components that capture maximum variance.^{[11] [12] [13] [14]}

2. Regularization (Lasso, Ridge, Elastic Net): Adds penalty terms to prevent large coefficients.^{[15] [16] [17] [18]}

3. Ensemble Learning: Combines multiple models to reduce variance (e.g., Random Forest, Bagging).^{[19] [20] [21]}

4. Cross Validation: Uses multiple train-test splits to ensure model generalizes well.^{[22] [23] [24]}

5. Dropout: Randomly deactivates neurons during training in neural networks.^{[25] [26]}

6. Business Knowledge: Understanding domain helps select relevant features and avoid spurious correlations[raw text].

2. Machine Learning Pipeline

The complete ML workflow consists of systematic steps[raw text]:

1. **Data Gathering:** Collect data from SQL queries, clients, or other teams
2. **Data Preparation:** Clean, preprocess, and split into features (X) and target (y)
3. **Train-Test Split:** Typically 70-30, 80-20, or 75-25 ratio

4. **Model Selection:** Choose appropriate algorithm based on problem type
5. **Training:** Fit model on training data
6. **Evaluation:** Assess performance using appropriate metrics
7. **Hyperparameter Tuning:** Optimize model parameters
8. **Prediction:** Deploy model for inference
9. **MLOps:** Monitor, maintain, and retrain as needed
10. **CI/CD Pipeline:** Automate deployment and updates

Key Distinction:

- **Data Analyst:** Works with structured data
- **Data Engineer:** Works with unstructured data, builds pipelines[raw text]

3. Regression Algorithms

3.1 Linear Regression

Simple Linear Regression: Models relationship between one independent variable and dependent variable[raw text].

$$y = mx + c$$

where m is slope and c is intercept.

Multiple Linear Regression: Extends to multiple independent variables[raw text].

$$y = m_1x_1 + m_2x_2 + m_3x_3 + c$$

Feature Selection Techniques:

- **Backward Elimination:** Start with all features, remove least significant
- **Recursive Feature Elimination (RFE):** Recursively removes features
- **P-value Analysis:** Statistical significance testing using statsmodels library[raw text]

3.2 Regularization Techniques

Regularization prevents overfitting by constraining model coefficients.^{[16] [17] [18] [15]}

Lasso Regression (L1):

$$\text{Loss} = \text{MSE} + \alpha \sum_{i=1}^n |w_i|$$

- Performs feature selection by driving some coefficients to exactly zero
- Creates sparse models
- More powerful for feature elimination[raw text]^[16]

Ridge Regression (L2):

$$\text{Loss} = \text{MSE} + \alpha \sum_{i=1}^n w_i^2$$

- Shrinks coefficients but doesn't eliminate features

- Better when all features are relevant
- Handles multicollinearity well ^[17] ^[16]

Elastic Net (L1 + L2):

$$\text{Loss} = \text{MSE} + \alpha_1 \sum |w_i| + \alpha_2 \sum w_i^2$$

- Combines benefits of both Lasso and Ridge
- Inherits feature selection from L1 and stability from L2 ^[18] ^[15] ^[17]

The **alpha** parameter controls regularization strength. Higher alpha means stronger regularization. ^[17] ^[16]

3.3 Gradient Descent Optimization

Gradient Descent iteratively updates model parameters to minimize loss function. ^[27] ^[28] ^[29] ^[30] ^[31]

Weight Update Formula:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w_{\text{old}}}$$

where η is the learning rate.

Types:

1. **Batch Gradient Descent:** Uses entire dataset for each update
 - Accurate gradient estimates
 - Slow for large datasets
 - Smooth convergence ^[30] ^[27]
2. **Stochastic Gradient Descent (SGD):** Uses single sample for each update
 - Fast updates
 - Noisy convergence path
 - Can escape local minima
 - Good for online learning ^[29] ^[31] ^[27] ^[30]
3. **Mini-Batch Gradient Descent:** Uses subset of data for each update
 - Balance between accuracy and speed
 - Leverages vectorization
 - Most commonly used in practice ^[28] ^[31]

3.4 Non-Linear Regression

Polynomial Regression: Increases degree of features to capture non-linear relationships.

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots$$

Support Vector Regressor (SVR):

- Uses hyperplane for regression with maximum margin distance

- Decision boundary defined by support vectors
- Formula: $-\epsilon < y - (wx + b) < +\epsilon$
- Maximizes margin between support lines for robust predictions

Decision Tree Regression: Uses tree structure with splits based on feature values.

Random Forest: Ensemble of decision trees using bagging. [20] [21] [32] [19]

XGBoost & LightGBM: Gradient boosting algorithms for high performance. [33] [34] [35] [36]

4. Classification Algorithms

4.1 Logistic Regression

Logistic regression applies sigmoid function to linear model for binary classification. [37] [38] [39] [40] [41]

Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where $z = wx + b$ [40] [37]

Key Properties:

- Output range: $[0, 1]$, interpretable as probability [5]
- S-shaped curve
- Handles outliers well due to asymptotic behavior
- Also called **MaxEnt classifier**

Why Sigmoid? It naturally emerges from Bayes' theorem for binary classification with Gaussian class conditionals, ensuring outputs represent valid probabilities. [38]

The **logit (log-odds)** function is the inverse of sigmoid:

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right) = wx + b$$
 [41]

This linear relationship makes the model interpretable and enables probabilistic predictions.

4.2 Support Vector Machine (SVM)

SVM finds optimal hyperplane that maximizes margin between classes. [42] [43] [44] [45]

Linear SVM: For linearly separable data, directly finds separating hyperplane.

Non-Linear SVM: Uses **kernel trick** to map data to higher dimensions where it becomes linearly separable. [44] [42]

Kernel Functions:

1. **Linear:** $K(x_i, x_j) = x_i \cdot x_j$
2. **Polynomial:** $K(x_i, x_j) = (x_i \cdot x_j + c)^d$

3. **RBF (Radial Basis Function):** $K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$

- Most commonly used
- Localized and finite response

4. **Sigmoid:** $K(x_i, x_j) = \tanh(\alpha(x_i \cdot x_j) + c)$ [43] [42] [44]

Key Concepts:

- **Hyperplane:** Decision boundary
- **Support Vectors:** Data points closest to hyperplane
- **Margin:** Distance between support vectors
- Goal: Maximize margin for better generalization[raw text] [45]

4.3 K-Nearest Neighbors (KNN)

KNN classifies based on majority vote of k nearest neighbors. [46] [47] [48]

Distance Metrics:

1. **Euclidean Distance** (L2-norm): Shortest straight-line distance

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$
 [47] [48] [46]

2. **Manhattan Distance** (L1-norm): Sum of absolute differences (taxicab distance)

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$
 [46] [47] [48]

3. **Minkowski Distance:** Generalization of both

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

- When $p=1$: Manhattan distance
- When $p=2$: Euclidean distance [48] [46]

4. **Cosine Distance:** Used for text/NLP, measures angle between vectors[raw text]

KNN Regression vs Classification:

- **Regression:** Takes mean of k nearest neighbors
- **Classification:** Takes majority class of k nearest neighbors[raw text]

Important Note: KNN is sensitive to outliers, unlike logistic regression which is robust due to sigmoid function[raw text].

4.4 Decision Tree Classification

Decision trees recursively split data based on features to create a tree structure. [49] [50] [51] [52]

Key Concepts:

Root Node: Starting point, selected based on maximum information gain[raw text].

Leaf Node: Terminal node representing final prediction (pure split when one class)[raw text].

Splitting Criteria:

1. Gini Index:

$$Gini = 1 - \sum_j p_j^2$$

- Measures impurity/misclassification probability
- Range: [0, 0.5]
- 0 indicates pure node^{[50] [51] [49]}

2. Entropy:

$$E = -\sum_{i=1}^n p_i \log_2(p_i)$$

- Measures randomness/disorder
- Range:^[5]
- 0 indicates pure node^{[51] [49] [50]}

Information Gain:

$$Gain = Entropy\{parent\} - Weighted\ Average\ Entropy\{children\}$$

- Feature with maximum gain becomes root node^[51]

Comparison: Gini is computationally faster than Entropy. Both work well, with Gini slightly preferred in practice.^{[49] [50]}

Pruning: Removing irrelevant branches to prevent overfitting^[raw text].

- **Pre-pruning:** Early stopping during tree growth
- **Post-pruning:** Remove branches after full tree is built

Hyperparameters:

- `max_depth`: Maximum tree depth
- `min_samples_split`: Minimum samples required to split
- `min_samples_leaf`: Minimum samples in leaf node^[raw text]

4.5 Random Forest

Random Forest is an ensemble of decision trees using **bagging** (Bootstrap Aggregating).^{[21] [32] [19] [20]}

How it Works:

1. Create multiple bootstrap samples (random sampling with replacement)
2. Train separate decision tree on each sample
3. For each tree, randomly select subset of features for splitting
4. Aggregate predictions: majority vote (classification) or average (regression)^{[32] [19] [20]}

Benefits:

- Reduces overfitting through averaging
- Reduces variance while maintaining low bias
- Handles large datasets efficiently

- Provides feature importance rankings^{[19] [21] [32]}

Key Parameters:

- `n_estimators`: Number of trees
- `max_features`: Number of features to consider for each split
- `max_depth`: Depth of each tree[raw text]

4.6 Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the "naive" assumption of feature independence[raw text].

Bayes Theorem:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

where:

- **Posterior:** $P(A|B)$ - probability of A given B
- **Likelihood:** $P(B|A)$ - probability of observing B given A
- **Prior:** $P(A)$ - initial probability of A
- **Marginal:** $P(B)$ - total probability of B[raw text]

Types of Naive Bayes:

1. **Gaussian Naive Bayes:** For continuous features following normal distribution
 - Assumes features are normally distributed
 - Uses mean and variance for probability calculation^{[53] [54] [55]}
2. **Multinomial Naive Bayes:** For discrete count data
 - Commonly used for text classification with word counts
 - Cannot handle negative values[raw text]^{[54] [53]}
3. **Bernoulli Naive Bayes:** For binary/boolean features
 - Works with presence/absence of features (0 or 1)
 - Good for document classification^{[56] [53] [54]}

Important Notes:

- Logistic Regression and Naive Bayes are probability-based algorithms[raw text]
- Decision trees and Naive Bayes don't require feature scaling[raw text]
- Fast training and prediction
- Works well with small datasets^[56]

4.7 Ensemble Methods: Boosting

XGBoost (Extreme Gradient Boosting):

- Improved version of Gradient Boosting
- Uses Taylor expansion for optimization
- Level-wise tree growth
- Regularization to prevent overfitting
- Highly efficient and parallelizable^{[34] [35] [33]}

LightGBM (Light Gradient Boosting Machine):

- Developed by Microsoft
- Leaf-wise (vertical) tree growth instead of level-wise
- Faster training through histogram-based algorithms
- Lower memory usage
- Can be 6-7x faster than XGBoost on large datasets^{[35] [36] [33]}

Key Differences:

- XGBoost: More robust, better for smaller datasets
- LightGBM: Faster, better for large datasets but can overfit more easily^{[33] [35]}

Boosting vs Bagging:

- **Bagging:** Trains models in parallel, reduces variance (Random Forest)^{[20] [19]}
- **Boosting:** Trains models sequentially, each correcting previous errors, reduces bias (XGBoost, LightGBM)^[34]

5. Performance Metrics for Classification

5.1 Confusion Matrix

The **confusion matrix** visualizes classification performance.^{[57] [58] [59] [60]}

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Derived Metrics:

Accuracy: Overall correctness

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Precision: Correctness of positive predictions

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall (Sensitivity/TPR): Coverage of actual positives

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Specificity: Coverage of actual negatives

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

F1-Score: Harmonic mean of precision and recall

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

When to Use:

- **Precision:** When false positives are costly (fraud detection, spam filtering)
- **Recall:** When false negatives are costly (disease diagnosis, security threats)
- **F1-Score:** When you need balance, especially with imbalanced data

5.2 ROC-AUC Curve

ROC (Receiver Operating Characteristic) Curve plots True Positive Rate vs False Positive Rate at various thresholds.

Components:

- **X-axis:** False Positive Rate (FPR) = 1 - Specificity
- **Y-axis:** True Positive Rate (TPR) = Sensitivity/Recall

AUC (Area Under Curve):

- Range: [0.5, 1.0]
- 0.5: Random guessing (diagonal line)
- 1.0: Perfect classification
- 0.8: Generally considered good performance

Interpretation: AUC represents the probability that the model ranks a random positive instance higher than a random negative instance.

Usage: ROC-AUC is particularly useful for:

- Comparing multiple models
- Evaluating performance across different thresholds
- Classification problems, especially with imbalanced data

6. Handling Imbalanced Data

Imbalanced datasets occur when one class significantly outnumbers others (e.g., 99% normal, 1% fraud).

6.1 Resampling Techniques

Oversampling: Increase minority class samples

- Simple duplication of minority samples
- Risk: Can lead to overfitting [\[66\]](#) [\[67\]](#) [\[65\]](#)

Undersampling: Decrease majority class samples

- Random removal of majority samples
- Risk: Loss of potentially useful information[raw text] [\[68\]](#)

SMOTE (Synthetic Minority Oversampling Technique):

- Creates synthetic samples using k-nearest neighbors
- Generates new samples along line segments connecting minority class instances
- Formula: $x' = x + \text{rand}(0,1) \times |x - x_k|$
- More sophisticated than simple duplication
- Reduces overfitting risk [\[67\]](#) [\[65\]](#) [\[66\]](#)

Implementation Strategy:

1. Apply SMOTE only on training data
2. Keep test data unchanged
3. Evaluate on original distribution [\[65\]](#) [\[66\]](#) [\[67\]](#)

Balanced vs Imbalanced:

- **Balanced:** Equal representation of all classes
- **Imbalanced:** Skewed class distribution requiring special techniques[raw text]

7. Model Evaluation and Tuning

7.1 Cross Validation

Cross Validation uses multiple train-test splits to ensure robust performance estimates. [\[23\]](#) [\[24\]](#) [\[22\]](#)

K-Fold Cross Validation:

- Splits data into k equal folds
- Each fold serves as test set once
- Final metric is average across all folds
- Reduces variance in performance estimates [\[24\]](#) [\[22\]](#) [\[23\]](#)

Stratified K-Fold:

- Maintains class distribution in each fold

- Critical for imbalanced datasets
- Ensures each fold is representative of overall data
- Prevents biased performance estimates^{[69] [22] [23] [24]}

Example: With 80% Class 0 and 20% Class 1, each fold maintains this 80-20 ratio.^{[23] [24]}

7.2 Hyperparameter Tuning

Grid Search CV:

- Exhaustive search over specified parameter grid
- Tests all combinations
- Computationally expensive but thorough[raw text]

Random Search CV:

- Random sampling from parameter distributions
- More efficient for large parameter spaces
- Often finds good solutions faster[raw text]

Best Practices:

- Use cross-validation during tuning to avoid overfitting
- Start with coarse grid, then refine
- Consider computational budget[raw text]

8. Clustering and Dimensionality Reduction

8.1 K-Means Clustering

K-Means partitions data into k clusters by minimizing within-cluster variance[raw text].

Algorithm:

1. Initialize k random centroids
2. Assign each point to nearest centroid (using Euclidean distance)
3. Recalculate centroids as mean of assigned points
4. Repeat until convergence

Choosing k: Use Elbow Method

- Plot WCSS (Within-Cluster Sum of Squares) vs number of clusters
- Choose k at "elbow" point where WCSS decrease slows[raw text]

WCSS Formula:

$$WCSS = \sum_{i=1}^k \sum_{x \in C_i} ||x - \mu_i||^2$$

where μ_i is centroid of cluster C_i [raw text]

8.2 Hierarchical Clustering

Hierarchical clustering builds tree-like structure of nested clusters[raw text].

Types:

1. **Agglomerative** (Bottom-up): Start with individual points, merge similar clusters
2. **Divisive** (Top-down): Start with one cluster, recursively split

Dendrogram: Visual representation showing hierarchical relationships and merge distances[raw text].

8.3 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) finds arbitrarily shaped clusters based on density[raw text].

Key Concepts:

- **Core Points:** Points with minimum number of neighbors within radius
- **Border Points:** Non-core points within radius of core point
- **Noise Points:** Points that don't belong to any cluster (outliers)[raw text]

Advantages:

- Doesn't require specifying number of clusters
- Can find arbitrary shapes
- Identifies outliers
- Robust to noise[raw text]

8.4 Principal Component Analysis (PCA)

PCA reduces dimensionality while preserving maximum variance.^{[12] [13] [14] [11]}

Process:

1. Standardize features
2. Compute covariance matrix
3. Calculate eigenvalues and eigenvectors
4. Select top k eigenvectors (principal components)
5. Project data onto new feature space^{[13] [14]}

Principal Components:

- PC1 captures maximum variance
- PC2 captures second-most variance (orthogonal to PC1)
- Each subsequent PC captures decreasing variance^{[11] [13]}

Explained Variance Ratio: Indicates how much information each PC retains[raw text].

Use Cases:

- Feature extraction
- Visualization (reduce to 2D/3D)
- Noise reduction
- Overcoming multicollinearity
- Speeding up training ^[12] ^[13] ^[11]

Important: PCA is unsupervised and used for both regression and clustering, not classification directly[raw text].

9. Natural Language Processing (NLP)

9.1 Text Representation Techniques

Bag of Words (BoW):

- Creates vocabulary of all unique words
- Represents documents as word frequency vectors
- Ignores word order and grammar
- Simple but effective baseline[raw text]

TF-IDF (Term Frequency-Inverse Document Frequency):

- Weighs words by importance
- Formula: $TF\text{-}IDF = TF \times IDF$
- **TF:** Frequency of word in document
- **IDF:** $\log\left(\frac{\text{Total Documents}}{\text{Documents with term}}\right)$
- Reduces weight of common words, emphasizes distinctive words[raw text]

9.2 Word Embeddings: Word2Vec

Word2Vec creates dense vector representations of words that capture semantic relationships.
^[70] ^[71] ^[72] ^[73]

Two Architectures:

1. CBOW (Continuous Bag of Words):

- Predicts target word from surrounding context words
- Input: Context words
- Output: Target word
- Faster training

- Better for smaller datasets
- Parameter: `sg=0[raw text]` [\[71\]](#) [\[72\]](#) [\[73\]](#) [\[70\]](#)

2. Skip-gram:

- Predicts context words from target word
- Input: Target word
- Output: Context words
- Better for rare words
- Captures more subtle relationships
- Parameter: `sg=1[raw text]` [\[72\]](#) [\[73\]](#) [\[70\]](#) [\[71\]](#)

Key Concepts:

- **Window Size:** Number of surrounding words to consider
- **Embedding Dimension:** Size of vector representation (typically 100-300)
- **Semantic Similarity:** Similar words have similar vector representations
- Example: $\text{vector}(\text{"king"}) - \text{vector}(\text{"man"}) + \text{vector}(\text{"woman"}) \approx \text{vector}(\text{"queen"})$ [\[70\]](#) [\[71\]](#)

Library: Gensim is commonly used for Word2Vec implementation[raw text].

Chunking: Splitting text into meaningful segments (noun phrases, verb phrases, prepositional phrases)[raw text].

10. Deep Learning: Artificial Neural Networks

10.1 Neural Network Fundamentals

Traditional Model: Input + Logic → Output

ML Model: Input + Output → Logic (learned from data)[raw text]

Components:

- **Input Layer:** Receives features
- **Hidden Layers:** Process information
- **Output Layer:** Produces predictions
- **Weights & Biases:** Learnable parameters
- **Activation Functions:** Introduce non-linearity [\[74\]](#) [\[75\]](#) [\[76\]](#)

10.2 Activation Functions

Activation functions introduce non-linearity, enabling networks to learn complex patterns. [\[75\]](#) [\[76\]](#) [\[77\]](#) [\[78\]](#) [\[74\]](#)

1. Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Range: (0, 1)
- S-shaped curve
- Used in output layer for binary classification
- **Drawback:** Vanishing gradient problem in hidden layers[raw text][76] [74]

2. TanH (Hyperbolic Tangent):

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Range: (-1, 1)
- Zero-centered (better than sigmoid)
- Used in output layer
- **Drawback:** Vanishing gradient in deep networks[raw text][74] [76]

3. ReLU (Rectified Linear Unit):

$$f(x) = \max(0, x)$$

- Range: [0, ∞)
- Most popular for hidden layers
- Fast computation
- Mitigates vanishing gradient
- **Drawback:** "Dying ReLU" problem (neurons can become inactive)[77] [78] [75] [76] [74]

4. Leaky ReLU:

$$f(x) = \max(0.01x, x)$$

- Range: (-∞, ∞)
- Solves dying ReLU problem with small negative slope
- Allows small gradient for negative inputs[raw text][78] [75] [76] [77]

5. Softmax (for output layer):

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- Converts scores to probabilities
- Used for multi-class classification
- Output sums to 1[raw text][76] [74]

Selection Guidelines:

- **Hidden layers:** ReLU or Leaky ReLU
- **Binary classification output:** Sigmoid
- **Multi-class output:** Softmax
- **Regression output:** Linear (no activation)[76]

10.3 Training Process

Forward Propagation:

1. Input data flows through network
2. Weighted sum calculated at each neuron: $z = wx + b$
3. Activation function applied: $a = f(z)$
4. Process repeats layer by layer until output
5. Loss calculated by comparing prediction to actual^{[79] [80] [81] [82]}

Backward Propagation:

1. Calculate gradient of loss with respect to output
2. Use chain rule to compute gradients layer by layer (backward)
3. Update weights: $w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$
4. Process moves from output to input layers^{[80] [81] [82] [79]}

One Epoch = One forward pass + One backward pass through entire training data.

Weight Update Formula:

$$w_{\text{new}} = w_{\text{old}} - \text{learning_rate} \times \frac{\partial \text{Loss}}{\partial w_{\text{old}}}$$

Weights are adjusted during backward propagation.

10.4 Common Challenges

Vanishing Gradient Problem:

- Gradients become extremely small in early layers
- Occurs when forward and backward propagation weights become equal
- Network stops learning effectively
- Common with sigmoid/tanh activations in deep networks^{[83] [84] [25]}

Solutions:

1. Use ReLU activation functions
2. Batch Normalization
3. Skip Connections (ResNets)
4. Proper weight initialization
5. Gradient clipping^{[84] [83] [25]}

Overfitting in Neural Networks:

Prevention Techniques:

1. **Dropout:** Randomly deactivate neurons during training (typical rate: 0.2-0.5)
 - Forces network to learn redundant representations

- Reduces co-dependency between neurons[raw text]^[26] ^[25]
- 2. **L1/L2 Regularization:** Add penalty for large weights[raw text]
- 3. **Early Stopping:** Monitor validation loss, stop when it starts increasing
- 4. **Batch Normalization:** Normalizes inputs to each layer, stabilizes learning^[83] ^[25]
- 5. **Data Augmentation:** Artificially increase training data

11. Convolutional Neural Networks (CNN)

CNNs are specialized for processing grid-like data, especially images.^[85] ^[86] ^[87] ^[88] ^[89]

11.1 CNN Architecture Layers

1. Convolution Layer:

- Applies filters/kernels to extract features
- Convolution operation merges two sets of information
- Creates feature maps
- **Stride:** Step size for filter movement (default: 1)
- **Padding:** Adds border to maintain spatial dimensions[raw text]^[87] ^[89] ^[85]

Mathematical Operation:

$$Output\{i,j\} = \sum_m \sum_n Input\{i+m,j+n\} \times Kernel\{m,n\}$$

2. Pooling Layer:

- Reduces spatial dimensions
- **Max Pooling:** Selects maximum value from each region
 - Retains most prominent features
 - Creates sharper images
- **Average Pooling:** Takes average value
- Benefits: Reduces parameters, computational cost, overfitting
- Provides translation invariance^[86] ^[88] ^[85] ^[87]

3. Flattening Layer:

- Converts 2D feature maps to 1D vector
- Prepares data for fully connected layers
- Doesn't alter information, just reshapes
- Required before passing to dense layers[raw text]^[88] ^[89] ^[85] ^[87]

4. Fully Connected Layer:

- Standard neural network layer
- Every neuron connected to all neurons in previous layer

- Performs final classification
- Uses activation function (softmax for multi-class)
- Makes final decision based on extracted features[raw text]^{[89] [85] [87] [88]}

Typical CNN Flow:

Input Image → Convolution → Activation (ReLU) → Pooling → Convolution → Activation → Pooling → Flatten → Fully Connected → Output^{[87] [89]}

11.2 Key Concepts

Feature Extraction: Early layers detect edges, later layers detect complex patterns (shapes, objects).^[87]

Dropout Layer: Added after dense layers to prevent overfitting in CNNs[raw text].^[85]

Why CNNs Work:

- **Parameter Sharing:** Same filter used across entire image
- **Sparse Connections:** Each neuron connected to small region
- **Hierarchical Learning:** Builds complex features from simple ones^[87]

Applications:

- Image classification
- Object detection
- Face recognition
- Medical image analysis
- Self-driving cars^[87]

12. MLOps and Deployment

MLOps (Machine Learning Operations) bridges data science and operations teams for production ML systems[raw text].

12.1 Key Components

- 1. Version Control:** Track code, data, and model versions
- 2. Continuous Integration:** Automated testing of ML code
- 3. Model Deployment:** Deploy models to production environment
- 4. Model Monitoring:** Track performance, detect drift
 - Performance checking
 - Retraining when necessary[raw text]
- 5. Model Update:** Automated retraining pipeline

12.2 MLOps Tools

- **MLflow**: Experiment tracking, model registry
- **Docker**: Containerization for reproducibility
- **TFX (TensorFlow Extended)**: End-to-end ML platform
- **Cloud Platforms**: AWS SageMaker, GCP AI Platform, Azure ML
- **Databricks**: Unified analytics platform
- **Kubeflow**: Kubernetes-based ML workflows[raw text]

12.3 Workflow

1. **Data Collection**: Gather data for training
2. **Model Training**: Train and validate model
3. **Model Deployment**: Deploy to production
4. **Model Monitoring**: Track performance metrics
5. **Model Update**: Retrain if performance degrades[raw text]

CI/CD Pipeline: Continuous Integration and Continuous Deployment automates the entire workflow from code commit to production deployment[raw text].

Practice Questions

Conceptual Questions

1. ANOVA and Metrics

- Q: Explain the relationship between SST, SSR, and SSE. Why is this decomposition useful?
- Q: When would you prefer Adjusted R-squared over R-squared?
- Q: Can R-squared be negative? Why or why not?

2. Bias-Variance

- Q: You have a model with 94% training accuracy and 34% test accuracy. What's the problem and how would you fix it?
- Q: Explain why ensemble methods like Random Forest reduce variance.
- Q: How does regularization help with the bias-variance tradeoff?

3. Algorithms

- Q: Compare Lasso, Ridge, and Elastic Net. When would you use each?
- Q: Why does logistic regression use sigmoid function instead of other functions?
- Q: Explain the difference between Batch GD, SGD, and Mini-batch GD with pros/cons.
- Q: When would you choose KNN over SVM for classification?

- Q: Why is Naive Bayes called "naive"? What assumption does it make?

4. Deep Learning

- Q: Explain vanishing gradient problem and how ReLU helps solve it.
- Q: Why use dropout? What's the typical dropout rate?
- Q: Describe the complete flow of data through a CNN for image classification.
- Q: What's the difference between CBOW and Skip-gram in Word2Vec?

5. Evaluation

- Q: You're building a cancer detection model. Should you optimize for precision or recall? Why?
- Q: When is accuracy not a good metric? Give examples.
- Q: Explain ROC curve and how to interpret AUC values.
- Q: Why use Stratified K-Fold instead of regular K-Fold for imbalanced data?

Coding Questions

1. Implement Linear Regression with Gradient Descent

```
import numpy as np

def gradient_descent_linear_regression(X, y, learning_rate=0.01, epochs=1000):
    """
    Implement linear regression using gradient descent

    Parameters:
    X: Features (n_samples, n_features)
    y: Target (n_samples,)
    learning_rate: Learning rate
    epochs: Number of iterations

    Returns:
    weights: Learned weights
    bias: Learned bias
    losses: Loss history
    """
    # Initialize weights and bias
    n_samples, n_features = X.shape
    weights = np.zeros(n_features)
    bias = 0
    losses = []

    for epoch in range(epochs):
        # Forward pass
        y_pred = np.dot(X, weights) + bias

        # Calculate loss (MSE)
        loss = np.mean((y_pred - y) ** 2)
        losses.append(loss)
```

```

# Calculate gradients
dw = (2/n_samples) * np.dot(X.T, (y_pred - y))
db = (2/n_samples) * np.sum(y_pred - y)

# Update parameters
weights -= learning_rate * dw
bias -= learning_rate * db

return weights, bias, losses

```

2. Implement K-Means Clustering

```

import numpy as np

def kmeans(X, k, max_iters=100):
    """
    Implement K-Means clustering

    Parameters:
    X: Data points (n_samples, n_features)
    k: Number of clusters
    max_iters: Maximum iterations

    Returns:
    centroids: Final centroids
    labels: Cluster assignments
    wcss: Within-cluster sum of squares
    """

    # Initialize centroids randomly
    n_samples = X.shape[0]
    random_indices = np.random.choice(n_samples, k, replace=False)
    centroids = X[random_indices]

    for _ in range(max_iters):
        # Assign points to nearest centroid
        distances = np.sqrt(((X - centroids[:, np.newaxis])**2).sum(axis=2))
        labels = np.argmin(distances, axis=0)

        # Update centroids
        new_centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])

        # Check convergence
        if np.allclose(centroids, new_centroids):
            break

        centroids = new_centroids

    # Calculate WCSS
    wcss = sum([np.sum((X[labels == i] - centroids[i])**2) for i in range(k)])

    return centroids, labels, wcss

```

3. Implement Decision Tree (Gini Index)

```

import numpy as np

def gini_index(groups, classes):
    """Calculate Gini index for a split"""
    n_samples = sum([len(group) for group in groups])
    gini = 0.0

    for group in groups:
        size = len(group)
        if size == 0:
            continue
        score = 0.0

        for class_val in classes:
            proportion = [row[-1] for row in group].count(class_val) / size
            score += proportion ** 2

        gini += (1.0 - score) * (size / n_samples)

    return gini

def test_split(index, value, dataset):
    """Split dataset based on feature and value"""
    left = [row for row in dataset if row[index] < value]
    right = [row for row in dataset if row[index] >= value]
    return left, right

def get_best_split(dataset):
    """Find best split point"""
    class_values = list(set(row[-1] for row in dataset))
    best_index, best_value, best_score = 999, 999, 999
    best_groups = None

    for index in range(len(dataset[0])-1):
        for row in dataset:
            groups = test_split(index, row[index], dataset)
            gini = gini_index(groups, class_values)

            if gini < best_score:
                best_index = index
                best_value = row[index]
                best_score = gini
                best_groups = groups

    return {'index': best_index, 'value': best_value, 'groups': best_groups}

```

4. Implement SMOTE for Imbalanced Data

```

import numpy as np
from sklearn.neighbors import NearestNeighbors

def smote(X, y, k=5, ratio=1.0):
    """
    Implement SMOTE for oversampling minority class

```



```

Parameters:
X: Features
y: Target labels
k: Number of nearest neighbors
ratio: Desired ratio of minority to majority class

Returns:
X_resampled, y_resampled: Resampled data
"""
# Identify minority class
unique, counts = np.unique(y, return_counts=True)
minority_class = unique[np.argmin(counts)]
majority_count = np.max(counts)
minority_count = np.min(counts)

# Calculate number of synthetic samples needed
n_synthetic = int(majority_count * ratio) - minority_count

# Get minority class samples
minority_indices = np.where(y == minority_class)[^0]
X_minority = X[minority_indices]

# Fit k-nearest neighbors
nn = NearestNeighbors(n_neighbors=k+1)
nn.fit(X_minority)

# Generate synthetic samples
synthetic_samples = []
for _ in range(n_synthetic):
    # Random sample from minority class
    idx = np.random.randint(0, len(X_minority))
    sample = X_minority[idx]

    # Find k nearest neighbors
    neighbors = nn.kneighbors([sample], return_distance=False)[^0][1:]

    # Choose random neighbor
    neighbor_idx = np.random.choice(neighbors)
    neighbor = X_minority[neighbor_idx]

    # Generate synthetic sample
    gap = neighbor - sample
    synthetic = sample + np.random.random() * gap
    synthetic_samples.append(synthetic)

# Combine original and synthetic data
X_resampled = np.vstack([X, synthetic_samples])
y_resampled = np.hstack([y, [minority_class] * n_synthetic])

return X_resampled, y_resampled

```

5. Implement Simple Neural Network with Backpropagation

```

import numpy as np

class SimpleNeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        # Initialize weights
        self.W1 = np.random.randn(input_size, hidden_size) * 0.01
        self.b1 = np.zeros((1, hidden_size))
        self.W2 = np.random.randn(hidden_size, output_size) * 0.01
        self.b2 = np.zeros((1, output_size))

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def sigmoid_derivative(self, x):
        return x * (1 - x)

    def forward(self, X):
        """Forward propagation"""
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = self.sigmoid(self.z2)
        return self.a2

    def backward(self, X, y, learning_rate=0.1):
        """Backward propagation"""
        m = X.shape[0]

        # Calculate gradients
        delta2 = self.a2 - y
        dW2 = np.dot(self.a1.T, delta2) / m
        db2 = np.sum(delta2, axis=0, keepdims=True) / m

        delta1 = np.dot(delta2, self.W2.T) * self.sigmoid_derivative(self.a1)
        dW1 = np.dot(X.T, delta1) / m
        db1 = np.sum(delta1, axis=0, keepdims=True) / m

        # Update weights
        self.W2 -= learning_rate * dW2
        self.b2 -= learning_rate * db2
        self.W1 -= learning_rate * dW1
        self.b1 -= learning_rate * db1

    def train(self, X, y, epochs=1000, learning_rate=0.1):
        """Train the network"""
        losses = []
        for epoch in range(epochs):
            # Forward pass
            output = self.forward(X)

            # Calculate loss
            loss = np.mean((output - y) ** 2)
            losses.append(loss)

            # Backward pass
            self.backward(X, y, learning_rate)

```

```
        if epoch % 100 == 0:
            print(f"Epoch {epoch}, Loss: {loss:.4f}")

    return losses
```

Project Ideas

1. Customer Churn Prediction

- Dataset: Telco customer data with demographics and usage patterns
- Tasks:
 - Handle imbalanced data using SMOTE
 - Compare Logistic Regression, Random Forest, XGBoost
 - Optimize for recall (don't miss churning customers)
 - Implement cross-validation
 - Deploy using Flask/FastAPI

2. Image Classification with CNN

- Dataset: CIFAR-10 or Fashion MNIST
- Tasks:
 - Build CNN architecture from scratch
 - Implement data augmentation
 - Use dropout and batch normalization
 - Compare different activation functions
 - Visualize learned filters

3. Sentiment Analysis

- Dataset: Movie reviews or Twitter data
- Tasks:
 - Implement BoW and TF-IDF
 - Train Word2Vec embeddings
 - Compare CBOW vs Skip-gram
 - Build classification model
 - Evaluate with precision, recall, F1

4. House Price Prediction

- Dataset: Kaggle House Prices
- Tasks:
 - Feature engineering

- Handle missing values
- Compare Linear, Ridge, Lasso, Elastic Net
- Use PCA for dimensionality reduction
- Implement Grid Search for hyperparameter tuning
- Calculate ANOVA metrics

5. Recommendation System

- Dataset: MovieLens or e-commerce data
- Tasks:
 - Implement K-Means for user clustering
 - Use PCA for visualization
 - Build collaborative filtering
 - Compare distance metrics
 - Evaluate using custom metrics

Key Takeaways

1. **Regression vs Classification:** Type 1/Type 2 errors for classification, R^2 /Adjusted R^2 for regression[raw text]
2. **Model Complexity:** Balance bias-variance through regularization, cross-validation, and ensemble methods[7] [8] [9]
3. **Algorithm Selection:**
 - Linear problems: Linear Regression, Logistic Regression
 - Non-linear: SVM with kernels, Decision Trees, Neural Networks
 - High-dimensional: Regularization (Lasso/Ridge), PCA
 - Imbalanced data: SMOTE, adjust class weights, optimize recall/precision[66] [65]
4. **Deep Learning Best Practices:**
 - Use ReLU for hidden layers, appropriate activation for output
 - Apply dropout (0.2-0.5) to prevent overfitting
 - Use batch normalization for deep networks
 - Monitor for vanishing gradients[25] [74] [83]
5. **Evaluation Strategy:**
 - Use appropriate metrics for problem type
 - Always use cross-validation
 - Consider business impact when choosing optimization metric[22] [23]
6. **NLP Progression:** BoW → TF-IDF → Word Embeddings (Word2Vec) → Transformers (future)[71] [70]

7. **Production ML:** MLOps ensures models remain effective over time through monitoring and retraining[raw text]

This comprehensive guide provides the foundation needed for ML/DL interviews and practical applications. Focus on understanding concepts deeply rather than memorizing formulas, and always relate techniques back to real-world problems.

**

1. <https://www.gs.washington.edu/academics/courses/akey/56008/lecture/lecture9.pdf>
2. <https://365datascience.com/tutorials/statistics-tutorials/sum-squares/>
3. <https://online.stat.psu.edu/stat462/node/107/>
4. <https://www.datacamp.com/tutorial/regression-sum-of-squares>
5. <https://onlinedegrees.sandiego.edu/artificial-intelligence-education/>
6. <https://analystprep.com/cfa-level-1-exam/quantitative-methods/analysis-of-variance-anova/>
7. <https://www.ibm.com/think/topics/overfitting-vs-underfitting>
8. <https://www.geeksforgeeks.org/machine-learning/underfitting-and-overfitting-in-machine-learning/>
9. <https://www.linkedin.com/pulse/mastering-model-accuracy-navigating-bias-variance-dilek-celik-phd-689ce>
10. <https://www.geeksforgeeks.org/machine-learning/ml-bias-variance-trade-off/>
11. <https://www.geeksforgeeks.org/machine-learning/reduce-data-dimensionality-using-pca-python/>
12. <https://www.ibm.com/think/topics/dimensionality-reduction>
13. <https://builtin.com/data-science/step-step-explanation-principal-component-analysis>
14. <https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/>
15. https://en.wikipedia.org/wiki/Elastic_net_regularization
16. <https://www.geeksforgeeks.org/machine-learning/lasso-vs-ridge-vs-elastic-net-ml/>
17. <https://www.mathworks.com/help/stats/lasso-and-elastic-net.html>
18. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3363152/>
19. <https://keylabs.ai/blog/random-forest-ensemble-learning-technique/>
20. <https://developers.google.com/machine-learning/decision-forests/random-forests>
21. <https://scikit-learn.org/stable/modules/ensemble.html>
22. <https://towardsdatascience.com/what-is-stratified-cross-validation-in-machine-learning-8844f3e7ae8e/>
23. <https://www.geeksforgeeks.org/machine-learning/stratified-k-fold-cross-validation/>
24. https://scikit-learn.org/stable/modules/cross_validation.html
25. <https://www.lunartech.ai/blog/mastering-the-vanishing-gradient-problem-advanced-strategies-for-optimizing-neural-networks>
26. <https://developers.google.com/machine-learning/crash-course/neural-networks/backpropagation>
27. <https://www.geeksforgeeks.org/machine-learning/difference-between-batch-gradient-descent-and-stochastic-gradient-descent/>
28. <https://www.baeldung.com/cs/gradient-stochastic-and-mini-batch>
29. <https://towardsdatascience.com/stochastic-gradient-descent-math-and-python-code-35b5e66d6f79/>

30. <https://sebastianraschka.com/faq/docs/gradient-optimization.html>
31. https://en.wikipedia.org/wiki/Stochastic_gradient_descent
32. <https://www.machinelearningmastery.com/bagging-and-random-forest-ensemble-algorithms-for-machine-learning/>
33. <https://neptune.ai/blog/xgboost-vs-lightgbm>
34. <https://www.geeksforgeeks.org/machine-learning/gradientboosting-vs-adaboost-vs-xgboost-vs-catboost-vs-lightgbm/>
35. <https://www.geeksforgeeks.org/machine-learning/lightgbm-vs-xgboost-which-algorithm-is-better/>
36. https://datamap.u.com/posts/classical_ml/gradient_boosting_variants/
37. <https://www.datacamp.com/tutorial/sigmoid-function>
38. <https://www.dailydoseofds.com/why-do-we-use-sigmoid-in-logistic-regression/>
39. <https://www.educative.io/answers/what-is-sigmoid-and-its-role-in-logistic-regression>
40. <https://developers.google.com/machine-learning/crash-course/logistic-regression/sigmoid-function>
41. <https://builtin.com/machine-learning/logistic-regression-classification-algorithm>
42. <https://data-flair.training/blogs/svm-kernel-functions/>
43. <https://www.geeksforgeeks.org/machine-learning/major-kernel-functions-in-support-vector-machine-svm/>
44. <https://www.freecodecamp.org/news/svm-kernels-how-to-tackle-nonlinear-data-in-machine-learning/>
45. <https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/>
46. <https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/>
47. <https://www.kdnuggets.com/2020/11/most-popular-distance-metrics-knn.html>
48. <https://www.geeksforgeeks.org/machine-learning/how-to-choose-the-right-distance-metric-in-knn/>
49. <https://quantdare.com/decision-trees-gini-vs-entropy/>
50. <https://www.geeksforgeeks.org/machine-learning/gini-impurity-and-entropy-in-decision-tree-ml/>
51. <https://towardsdatascience.com/decision-trees-explained-entropy-information-gain-gini-index-ccp-pruning-4d78070db36c/>
52. <https://www.datasciencewizards.ai/getting-started-with-machine-learning-algorithms-decision-trees/>
53. <https://www.geeksforgeeks.org/machine-learning/bernoulli-naive-bayes/>
54. <https://stackoverflow.com/questions/48580762/difference-of-three-naive-bayes-classifiers>
55. https://en.wikipedia.org/wiki/Naive_Bayes_classifier
56. <https://towardsdatascience.com/bernoulli-naive-bayes-explained-a-visual-guide-with-code-examples-for-beginners-aec39771ddd6/>
57. <https://towardsdatascience.com/performance-metrics-confusion-matrix-precision-recall-and-f1-score-a8fe076a2262/>
58. <https://www.datacamp.com/tutorial/what-is-a-confusion-matrix-in-machine-learning>
59. <https://towardsdatascience.com/confusion-matrix-made-simple-accuracy-precision-recall-f1-score/>
60. https://en.wikipedia.org/wiki/Precision_and_recall
61. <https://www.geeksforgeeks.org/machine-learning/auc-roc-curve/>
62. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10664195/>
63. <https://www.medcalc.org/en/manual/roc-curves.php>

64. <https://www.indianpediatrics.net/apr2011/277.pdf>
65. <https://www.geeksforgeeks.org/machine-learning/ml-handling-imbalanced-data-with-smote-and-near-miss-algorithm-in-python/>
66. <https://www.machinelearningmastery.com/smote-oversampling-for-imbalanced-classification/>
67. <https://www.geeksforgeeks.org/machine-learning/handling-imbalanced-data-for-classification/>
68. <https://towardsai.net/p/artificial-intelligence/handling-imbalanced-datasets-in-machine-learning-smote-oversampling-undersampling-explained>
69. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html
70. <https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp-comparison-between-cbow-and-skip-gram-models/>
71. <https://kavita-ganesan.com/comparison-between-cbow-skipgram-subword/>
72. <https://www.youtube.com/watch?v=UqRCEmrV1gQ>
73. <https://www.geeksforgeeks.org/nlp/continuous-bag-of-words-cbow-in-nlp/>
74. <https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/>
75. <https://www.geeksforgeeks.org/deep-learning/choosing-the-right-activation-function-for-your-neural-network/>
76. <https://www.v7labs.com/blog/neural-networks-activation-functions>
77. <https://www.kaggle.com/code/avadhutvarvatkar/activations-functions-in-deep-learning>
78. https://ml-cheatsheet.readthedocs.io/en/latest/activation_functions.html
79. <https://www.datacamp.com/tutorial/forward-propagation-neural-networks>
80. <https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/>
81. <https://www.linkedin.com/pulse/understanding-forward-backward-propagation-neural-suresh-beekhani-e0rkf>
82. http://d2l.ai/chapter_multilayer_perceptrons/backprop.html
83. <https://www.geeksforgeeks.org/deep-learning/vanishing-and-exploding-gradients-problems-in-deep-learning/>
84. https://www.reddit.com/r/MachineLearning/comments/mup9ok/d_why_solving_the_vanishing_gradients_problem/
85. <https://www.upgrad.com/blog/basic-cnn-architecture/>
86. <https://www.geeksforgeeks.org/deep-learning/cnn-introduction-to-pooling-layer/>
87. <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>
88. <https://www.superdatascience.com/blogs/convolutional-neural-networks-cnn-step-3-flattening>
89. <https://www.linkedin.com/pulse/convolutions-pooling-flattening-pranav-kumar-pb>
90. <https://www.coursera.org/resources/ai-terms>
91. <https://www.pearson.com/international-schools/international-schools-blog/2023/12/ai-for-education--a-glossary-of-ai-terms-for-teachers.html>
92. <https://www.powerschool.com/blog/ai-in-education/>
93. <https://www.oneusefulting.org/p/how-to-use-ai-to-teach-some-of-the>
94. <https://www.edutopia.org/article/7-ai-tools-that-help-teachers-work-more-efficiently/>
95. <https://internationalschools.britishcouncil.org/blog/ai-in-education-how-to-use-ai-tools-in-your-classroom-next-term>

96. https://www.reddit.com/r/AskStatistics/comments/f3lbkw/euclidean_distance_vs_manhattan_distance_for_knn/
97. <https://www.youtube.com/watch?v=lZW7U0Ksiwg>
98. <https://www.kaggle.com/code/residentmario/undersampling-and-oversampling-imbalanced-data>